外文翻译

原文信息

- **原文标题**：EmbodiedRAG: Dynamic 3D Scene Graph Retrieval for Efficient and Scalable Robot Task Planning
- **作者**：Meghan Booker, Gerry Chen, Kartik Saigirdar, Mariana Bernardes Martins, Elijah Stanger-Jones, Jiuguang Wang, Seth Hutchinson
- **来源**：arXiv:2410.23968, 2024（Johns Hopkins University Applied Physics Laboratory / Johns Hopkins University）
- **译者**：[姓名]

译文

EmbodiedRAG：面向高效可扩展机器人任务规划的动态 3D 场景图检索

摘要

大语言模型（LLM）的最新进展为机器人在真实开放世界环境中的任务规划提供了令人振奋的推动力。3D 场景图（3DSG）作为一种紧凑且语义丰富的环境表征，为基于 LLM 的规划器提供了有前景的环境锚定（grounding）方案。然而，随着机器人操作环境规模的扩大（例如跟踪的实体数量增加）以及场景图信息复杂度的提升（例如维护更多属性），将 3DSG 原样提供给基于 LLM 的规划器很快变得不可行，原因在于 LLM 的输入 token 数量限制以及 LLM 中存在的注意力偏差。

受检索增强生成（RAG）方法成功经验的启发——RAG 方法通过检索与查询相关的文档片段来辅助 LLM 的问答——我们将这一范式适配到具身智能领域。具体而言，我们提出了一个 3D 场景子图检索框架，称为 EmbodiedRAG，用于增强基于 LLM 的规划器以执行自然语言机器人任务。值得注意的是，我们检索的子图能够适应环境的变化以及任务相关性的变化。我们在 AI2Thor 模拟家庭任务中验证了 EmbodiedRAG 显著降低输入 token 数量（降低一个数量级）和规划时间（平均每步规划时间减少高达 70%）的能力，同时提高了单臂移动操作机器人的任务成功率。此外，我们在配备操作臂的四足机器人上实现了 EmbodiedRAG，以展示其在真实环境边缘部署中的性能优势。

1 引言

3D 场景图（3DSG）（例如 [1, 2, 3, 4]）和基于大语言模型（LLM）的规划（例如 [5, 6, 7, 8, 9, 10]）领域近期令人振奋的进展，正在推动机器人任务规划与执行向丰富的大规模环境和复杂的长时间跨度任务扩展。特别是，3D 场景图使机器人能够维护其操作环境的丰富、语义化且紧凑的表征，该表征可以被序列化后供基于 LLM 的规划器使用。因此，以语言作为桥梁，我们可以将基于 LLM 的规划器锚定在机器人的操作环境中，利用 LLM 的优势（如常识推理）进行有效规划。

然而，随着机器人环境规模的扩大（例如从房间扩展到房屋再到办公楼）或实体信息复杂度的增加（例如维护更多属性或每个实体的详细描述），将 3DSG 原样提供给基于 LLM 的规划器会带来若干挑战。首先，3DSG 几乎会立即占满 LLM 的输入 token 限制（例如 GPT-4o 的 128k token 限制）。即使输入 token 数量不是限制因素，减少输入 token 数量也具有加速机器人规划时间的重要潜力。其次，LLM 的注意力能力存在不足：研究表明 LLM 容易被输入提示中与任务无关的信息分散注意力 [11, 12]，并且对输入提示开头或结尾位置的信息存在位置偏差 [13, 14]。这些限制使得 LLM 难以识别完成任务所需的重要实体和环境变化。移除与任务无关的信息还可以消除 3DSG 中相关的不确定性，从而可能产生更鲁棒的规划。

为缓解这些挑战，我们从检索增强生成（RAG）方法的成功经验中获得启发（综述见 [15]），RAG 方法通过检索与任务相关的文档片段来辅助 LLM 的问答，我们将这一范式适配到具身智能领域。具体而言，我们寻求从机器人的 3DSG 中检索与任务相关的子图，即仅包含足以完成任务的实体、属性和关系。重要的是，检索到的子图应能适应环境的变化和任务相关性的变化。例如，如果新的任务相关信息被添加到 3DSG 中，提供给 LLM 的子图应纳入这些新信息。此外，在重规划过程中，基于 LLM 的规划器可能会意识到不同的实体可以帮助完成任务（即检索到的子图应纳入规划器的反馈）。

贡献声明。 我们的主要贡献是一个 3D 场景子图检索框架，称为 EmbodiedRAG，用于增强基于 LLM 的机器人任务规划器。我们的关键创新在于将 RAG 范式适配到具身智能领域，提出了任务相关实体和属性的预检索方法、3DSG 索引方法、检索与 3DSG 锚定方法、计划生成方法以及纳入规划器反馈的检索方法。据我们所知，这是首个为基于 LLM 的规划提取相关子图的工作，它能够适应时变场景图（例如 3DSG 可以在线构建），不需要 LLM 解析或操作整个 3DSG 进行规划，并且不依赖于特定的 3DSG 格式（例如层级结构、开放词汇、属性类型）。我们展示了 EmbodiedRAG 在 AI2Thor 模拟家庭任务中显著降低输入 token 数量和规划时间（累计 token 使用减少高达 90%，平均每步规划时间减少 70%）的能力，同时提高了规划质量和成功率。此外，我们在配备操作臂的四足机器人平台上实现了 EmbodiedRAG，并与使用完整 3DSG 的智能体进行了对比（后者通常会立即被大量 token 淹没）。

2 相关工作

任务驱动表征。 任务驱动表征旨在为给定任务充分概括机器人的状态和环境（即机器人拥有足够的信息来成功完成任务）。人类在运用此类表征方面已有充分研究（例如注视启发式 [16, 17]），以获得对环境中与任务无关干扰物的强鲁棒性、实时决策的规划效率提升以及对新任务的泛化能力 [18, 19]。机器人领域的各种方法试图利用信息瓶颈 [20, 2, 21]、最小化以任务为条件的表征规模复杂度 [22, 23]，或利用

马尔可夫决策过程的状态抽象理论 [24, 25, 26] 来构建或学习任务驱动表征。这些方法的挑战在于向复杂环境和任务的可扩展性；通常需要先验的环境或任务知识。相比之下，我们的框架能够快速泛化到不同的自然语言任务，并在线适应任务相关性的变化，无需先验的机器人操作环境信息。

检索增强生成。 检索增强生成（RAG）[27] 最初被引入以高效检索与查询相关的文档片段，用领域特定或最新信息增强 LLM 以正确回答查询。此后，许多技术和适配方案被提出，如预检索查询重写 [28]、迭代知识检索 [29] 以及向知识图谱结构的扩展 [30]。与经典 RAG 公式不同，我们在具身智能领域中操作，机器人有能力改变被检索的信息源（即影响环境），信息源可能是时变的，且对原始查询重要的信息可能随着机器人执行计划而改变。

基于 3D 场景图的规划。 近期使用 3DSG 的规划方法依赖文本和图像嵌入（如 SentenceBERT [31]、CLIP [32]）来检索与自然语言任务或子任务相关的实体 [3, 2, 33]。Gu 等人 [3] 表明仅使用 CLIP 嵌入会遗漏任务中的功能可供性或否定表达，并建议使用 LLM 直接从 3DSG 中提取实体；但随着 3DSG 规模增长，这种方法将变得不可行。Chang 等人 [33] 提出通过嵌入进行子图匹配的实体检索。总体而言，这些方法聚焦于物体定位问题（即检索与自然语言任务相关的单个实体），而忽略了捕获多个实体、特定状态或属性信息以及更复杂任务规划所需的关系。

一些方向旨在通过基于学习的方法 [34] 或图剪枝策略 [35] 找到所有任务相关实体。与我们工作最相似的是 SayPlan [6]，它通过允许 LLM 在规划过程中展开和折叠 3DSG 中的节点来为基于 LLM 的规划器提供压缩的 3DSG。然而，压缩效果与所提供 3DSG 的层级结构绑定，因此在底层（例如房间内的所有物体可能并非都与任务相关）的压缩效果有限。此外，他们的方法假设可以访问预构建的 3DSG。我们的工作开发了一种检索多个任务相关实体、实体特定状态或属性信息及其关系的方法，无需 LLM 解析大型 3DSG 或先验访问 3DSG。

3 问题形式化

我们的目标是设计一个子图提取框架来增强基于 LLM 的机器人规划器，该框架需要：(i) 提取最小化且任务相关的子图，以扩展到大规模环境并适应 LLM 输入 token 限制；(ii) 适应时变的 3DSG（即不需要先验已知的 3DSG）；(iii) 不依赖于特定的 3DSG 格式（例如不要求层级结构或特定属性）。

为形式化这一问题，我们将机器人操作的环境 $\mathcal{E}$ 建模为部分可观测马尔可夫决策过程（POMDP），其中在时刻 t 的状态 s_t 下，给定自然语言任务 l，机器人根据策略 π 执行动作 $a_t \in \mathcal{A}$，并转移到新状态 s_{t+1}。我们假设机器人正在构建和维护一个语义化表征观测环境状态的 3DSG，$\mathcal{G}_t = \langle O_t, E_t \rangle$，其中 $O_t = \{o_j\}$（j=1…J）和 $E_t = \{e_m\}$（m=1…M）分别是给定时间步的实体（如物体）集合和边集合。对于每个实体 o_j，其信息可以作为属性 $C_{j,t}^{attr} = \{c_{j,i}\}$（i=1…I）存储（例如属性、动作可供性、图像嵌入等），类似地，边的属性 $C_{m,t}^{edge} = \{c_{m,f}\}$（f=1…F）可以捕获实体间关系（例如描述"鸡蛋通常在冰箱里"，或特定关系类型的属性，o_1 在 o_2 上面）。

我们进一步假设策略 π 由提供了 $\mathcal{G}_t$ 和上下文窗口 x_t 的 LLM 生成，上下文窗口包含自然语言任务 l 和任何额外提示 p。增强的上下文窗口 $\tilde{x}_t := \{l, p, \mathcal{G}_1, a_1, \ldots, \mathcal{G}_{t-1}, a_{t-1}\}$ 包含过去的 LLM 规划历史，可以提供

给 LLM。为了向基于 LLM 的规划器提供子图而非完整 3DSG，我们需要一个提取策略 $f: (\mathcal{G}_t, \tilde{x}_t) \mapsto \mathcal{G}'_t$，其中 $\mathcal{G}'_t = \langle O'_t, E'_t \rangle$ 是提取的子图，即 $O'_t \subseteq O_t$ 且 $E'_t \subseteq E_t$。因此，挑战在于开发一个良好的时变提取策略，使其提供的子图包含足够的实体、实体特定属性和关系，以成功推理当前任务 l。我们在后续章节中讨论所提出的方法 EmbodiedRAG。

4 EmbodiedRAG 方法

为提取任务相关子图，我们的关键洞察是将问题与 RAG 范式相连接。RAG 框架通常包含文档索引阶段（将信息分块并嵌入为向量表示）、检索阶段（检索与用户查询最相关的文档）以及生成阶段（LLM 使用检索到的文档回答查询）[27, 15]。更高级的 RAG 框架还使用预检索阶段，通过查询转换或重写等策略来提升检索文档的质量 [28]。在以下小节中，我们讨论这些 RAG 阶段如何启发我们的子图提取策略 $f(\mathcal{G}_t, \tilde{x}_t)$，以及我们如何纳入反馈机制以在真实环境中运行（见图 1）。

4.1 3DSG 的文档索引

为了在 RAG 范式中工作，我们首先需要将文档的概念与 3DSG 相连接。通常，RAG 框架中的一个重要考虑是如何最佳地分块文档内容（例如句子、段落、页面、图像）。3DSG 在构建时已经天然地按实体分块，因此一个自然的形式化方式是将每个实体 o_j 视为待检索的文档。具体而言，我们将物体类别或开放词汇标签指定为文档的页面内容。属性 $C_{j,t}^{attr}$ 被指定为文档的元数据。或者，属性可以被视为子文档，当它们被检索时，父文档 o_j 也被检索。后者在属性不够简洁的情况下（例如长描述）更为可取。然后，每个文档被嵌入到向量存储中以供下游任务高效检索。开源的本地向量存储如 Chroma [36] 和 FAISS [37] 可以在 LangChain [38] 等 LLM 集成框架中使用。

4.2 LLM 引导的预检索抽象

我们借鉴的下一个 RAG 组件是预检索阶段。在我们的设置中，我们希望避免让 LLM 解析或遍历整个 3DSG，而且在机器人部署初期，3DSG 可能尚不可用。因此，我们需要一种策略，能够在关于实体的知识变得可用时提升检索实体的质量。我们提出从自然语言任务 l 和将在 3DSG 中维护的属性生成面向实体的抽象。具体而言，我们在规划之前且不具备任何环境知识（即无法访问 $\mathcal{G}_t$）的情况下，要求 LLM 建议一组与自然语言任务 l 相关的实体 $\{\hat{o}_r\}$（r=1…R）。如果预期在 3DSG 中跟踪属性（这是机器人部署前已知的设计选择），则对于每个实体 $\hat{o}_r$，我们要求 LLM 建议哪些属性是重要的。

为说明这种预检索方法的优势，考虑一个家庭机器人被要求为人类煮鸡蛋的场景。直觉上，机器人至少需要一个鸡蛋、一个烹饪锅或容器以及一个加热源来完成此任务。这种快速抽象可以直接从任务描述中推断，能够处理否定表达、功能可供性和多个实体，且不需要关于其操作的家庭环境的显式知识（例如物体位置、厨房几何形状、可用物体选项等）。抽象中的实体甚至可能不存在于机器人的操作环境中，但它们为应该检索的内容提供了强有力的初始方向。如果 3DSG 预期维护属性 {温度, 可破碎, 可切换}，则给定任务中与鸡蛋相关的属性仅为 {温度, 可破碎}，而与锅相关的属性为 {温度}。

4.3 子图 Grounding 检索

当机器人开始在其操作环境 $\mathcal{E}$ 中部署时，3DSG 构建开始，每个实体被嵌入为文档。为构建子图 $\mathcal{G}'_t$，我们首先检索与预检索实体 {ô_r} 相似度最高的前 k 个文档（经阈值过滤）。检索到的实体作为进入 3DSG 的入口点，类似于 GraphRAG [30] 中局部搜索组件获取知识图谱访问权限的方式。接下来，我们从这些节点取诱导子图以捕获检索实体之间的关系。如果这些是成功完成任务所需的唯一实体，则基于 LLM 的规划器只需推理它们之间的关系。我们引入了一个反馈机制（在 4.5 节描述）来处理 O'_t 不包含完成环境 $\mathcal{E}$ 中任务所需的所有实体的场景（例如遮挡、任务相关性变化，或由于不良预检索导致的缺失实体或属性）。如果存在与预检索实体 ô_r 关联的属性，则这些属性被应用于 ô_r 的检索实体集合。我们将此带有过滤后任务相关属性的诱导子图 $\mathcal{G}'_t$ 序列化为 JSON 格式，供基于 LLM 的规划器使用。

4.4 计划生成

RAG 框架的最后一个主要组件是生成阶段，其中原始查询和检索到的文档被提供给 LLM 以回答查询。在这里，我们希望 LLM 生成动作 a_t 而非查询的答案，即作为我们的策略 $\pi(a_t \mid \mathcal{G}'_t, \tilde{x}_t)$。为此，我们利用 ReAct 风格的 LLM 智能体 [9]，该智能体被设计为根据过去动作和观测的历史生成动作。值得注意的是，动作空间 $\mathcal{A}$ 被语言增强，以提供关于推理和未来动作的思考 τ_t，而不影响环境。因此，在每个时间步 t，我们向 ReAct 风格的 LLM 智能体提供其上下文历史 $\tilde{x}_t = \{l, p, \mathcal{G}'_1, (a_1, \tau_1), \ldots, \mathcal{G}'_{t-1}, (a_{t-1}, \tau_{t-1})\}$ 和更新后的子图 $\mathcal{G}'_t$ 作为观测。

4.5 反馈机制

到目前为止，检索过程是开环的，遵循经典 RAG 框架。然而，基于 LLM 的规划器的增强动作 (a_t, τ_t) 能够以两种方式闭合反馈回路并影响 EmbodiedRAG 框架中的组件。

直觉上，动作 a_t 直接影响环境，进而更新文档或添加新文档。这个反馈回路对于基于 LLM 的规划器验证其动作是否按预期完成至关重要（例如，如果 LLM 将鸡蛋打在锅上，则看到鸡蛋的温度属性升高），或向规划器展示新发现的任务相关实体（例如，基于 LLM 的规划器一直在寻找鸡蛋，而鸡蛋最近才被填充到完整 3DSG 中）。

第二个反馈组件通过思考 τ_t 实现。虽然思考不影响环境，但我们的关键洞察是利用它们来影响检索机制，因为它们提供了关于基于 LLM 的规划器当前正在做出、希望做出或计划做出决策的信息类型（例如特定类型的物体或属性）的洞察。为处理思考 τ_t，我们使用 RAG 的自查询（Self-Query）方法（例如 [39]），该方法使用 LLM 从原始提示生成结构化查询，以识别重要语义并执行元数据过滤。我们不传入原始任务，而是传入思考以识别关键实体及其属性（即要过滤的元数据）。例如，在煮鸡蛋时，基于 LLM 的规划器可能意识到需要一个翻转鸡蛋的工具——一个不在原始预检索抽象中的实体。自查询机制随后会提议检索与"翻转工具"相关的实体。在检索时，如果铲子或翻转器等实体存在于 $\mathcal{G}_t$ 中，它们将被添加到 $\mathcal{G}'_t$。这种反馈还可以帮助检索与遮挡或障碍物相关的实体，以及随着任务执行而变得相关的元数据变化。

5 仿真实验

我们在四个 AI2Thor 模拟器 [40] 厨房场景中评估了 EmbodiedRAG 在模拟家庭任务中的效果（图 4）。任务范围从 20 个简单任务（例如"把茶壶放在炉子上"）到 20 个困难任务（例如"煮鸡蛋"和"将所有非刀具的厨房用具放入抽屉"）。单臂移动操作机器人有 40 次迭代来完成任务，可以执行面向物体的动作，如 pickup(object) 或 placeon(object)。

具体而言，我们研究了 EmbodiedRAG 在两种不同场景下对输入 token 数量、规划效率和成功率的影响：(i) 存在与任务无关的干扰实体；(ii) 存在大量属性。对于每种场景，我们将方法与两个基线进行比较。第一个基线是简单的 ReAct 智能体 [9]，通过显式动作请求 getdiscoveredobjects() 获取环境知识。第二个基线 Full-Mem 在每个规划时间步被提供完整的 3DSG $\mathcal{G}_t$ 作为观测。我们的方法 EmbodiedRAG 仅被提供检索到的子图 $\mathcal{G}'_t$ 作为观测，但可以通过显式请求访问 $\mathcal{G}_t$。我们还引入了 EmbodiedRAG-strict，它在任何时候都无法访问 $\mathcal{G}_t$。

5.1 干扰实体

为模拟大规模环境以及 3DSG 构建过程中可能出现的物体存在不确定性，我们研究了 3DSG 中不存在的干扰物体的影响。为了扩展到大量干扰物体并避免超出输入 token 限制，我们仅向基于 LLM 的规划器呈现 $\mathcal{G}_t$ 中的实体集合（即无属性或边）。我们使用 290 和 1,135 个干扰物体评估了上述智能体，并在图 2 中展示了两种不同 LLM（llama3.1:70b 和 GPT-4o-mini）的结果。

总体而言，EmbodiedRAG 和 EmbodiedRAG-strict 相比基线展示了若干重要改进。两者都能够使用 GPT-4o-mini 持续完成多个困难任务，并在显著降低平均每步规划时间和累计 token 使用量的同时总体保持成功率。此外，EmbodiedRAG-strict 在所有三个指标上的性能不随干扰物体数量增加而下降，而所有其他智能体都受到增加的影响。这突显了我们方法与基于 LLM 的规划器配合使用时的可扩展性。

5.2 带干扰属性的完整 3DSG

接下来，我们运行相同的任务，但构建以 JSON 格式提供给基于 LLM 的规划器的 3DSG，包含 28 种可能的属性以及表示空间和控制关系的实体间边。我们还实现了 EmbodiedRAG-feedback 以纳入 4.5 节讨论的自查询检索机制。结果在表 1 的仿真部分展示（使用 GPT-4o-mini）。

值得注意的是，EmbodiedRAG-strict 在规划时间和累计 token 方面表现最佳，且无需访问完整 3DSG。ReAct 智能体具有可比的成功率，但使用了超过 2.5 倍的 token 和更多时间来推理观测。纳入自查询检索提高了 EmbodiedRAG-strict 的成功率，同时保持了较低的 token 数量，这证明了反馈在子图检索过程中的重要性。特别是，智能体能够获取最初与任务无关但在操作环境中重规划期间变得相关的物体和属性。

6 硬件实验

此外，我们展示了 EmbodiedRAG 在真实环境中四足机器人上运行的效果。我们为 Boston Dynamics 的 Spot 机器人配备了 NVIDIA Jetson AGX Orin 和 Zed 2 RGB-D 相机，以使用 ConceptGraphs [3] 进

行 3DSG 构建。我们还在机载设备上使用 llama3.1:8b 运行基于 LLM 的规划器。与第 5 节中的简单任务类似，我们在两种不同环境规模下为 Spot 机器人分配移动操作任务。第一种情况下，机器人的 3DSG 仅包含立即可见的少量实体（小环境）；第二种情况下，我们让机器人映射一个大型实验室和办公空间（大环境）。我们为每个智能体（Full-Mem 和 EmbodiedRAG-strict）在每种环境规模下运行 5 次试验，共 20 次试验。结果在图 3 和表 2 中展示。

两个智能体在小环境中各成功完成了 10 次试验中的 1 次。然而，所提供计划的质量存在显著差异，特别是在失败模式方面（图 3）。当从较小环境切换到较大环境时，Full-Mem 的幻觉数量增加了一倍多，从多步规划退化为仅完成一步就发生灾难性幻觉。与此同时，EmbodiedRAG-strict 没有产生幻觉，而是由于映射更大空间导致感知错误增加。重要的是，环境规模的增加并未影响智能体的规划和尝试恢复能力，这从两种环境规模中失败前采取的相似步数可以看出。

7 结论

我们开发并评估了 EmbodiedRAG，以解决在动态开放世界环境中将 3D 场景图与 LLM 高效集成用于机器人任务规划的挑战。值得注意的是，我们的方法不需要先验的 3DSG，并且避免向基于 LLM 的规划器提供整个 3DSG，从而实现更快的规划生成和更高的成功率（与仅使用 ReAct 的智能体和接收完整 3DSG 的智能体相比）。对我们方法至关重要的是反馈组件，它有助于部署期间的规划恢复。基于 LLM 的规划器建议的动作影响环境，而规划器的自然语言思考影响额外的任务相关检索。

局限性与未来工作。 EmbodiedRAG 检索的实体依赖于添加到 3DSG 中的实体，因此性能可能受到感知不确定性（例如同一物体的多个实体表示将导致多个实体被检索）和定位不确定性（例如机器人可能移动到相对于检索实体的错误位置）的影响。未来方向可以包括扩展到多模态检索技术——我们的框架已具备扩展到该方向的能力。我们的框架还可以受益于更快的结构化反馈查询；本工作中使用的自查询机制尽管提高了成功率，但几乎使每步规划时间翻倍。特别是，我们希望我们的框架能够促进针对具身智能领域的预检索、检索和自查询组件的进一步研究。

原文节选

EmbodiedRAG: Dynamic 3D Scene Graph Retrieval for Efficient and Scalable Robot Task Planning

Meghan Booker, Gerry Chen, Kartik Saigirdar, Mariana Bernardes Martins, Elijah Stanger-Jones, Jiuguang Wang, Seth Hutchinson

Johns Hopkins University Applied Physics Laboratory / Johns Hopkins University

arXiv:2410.23968, 2024

Abstract

Recent advances in Large Language Models (LLMs) have helped facilitate exciting progress for robotic planning in real, open-world environments. 3D scene graphs (3DSGs) offer a promising environment representation for grounding such LLM-based planners as they are compact and semantically rich. However, as the robot's environment scales (e.g., number of entities tracked) and the complexity of scene graph information increases (e.g., maintaining more attributes), providing a 3DSG as-is to an LLM-based planner quickly becomes infeasible due to input token count limits and attentional biases present in LLMs.

Inspired by the successes of Retrieval-Augmented Generation (RAG) methods that retrieve query-relevant document chunks for LLM question and answering, we adapt the paradigm for our embodied domain. Specifically, we propose a 3D scene subgraph retrieval framework, called EmbodiedRAG, that we augment an LLM-based planner with for executing natural language robotic tasks. Notably, our retrieved subgraphs adapt to changes in the environment as well as changes in task-relevancy as the robot executes its plan. We demonstrate EmbodiedRAG's ability to significantly reduce input token counts (by an order of magnitude) and planning time (up to 70% reduction in average time per planning step) while improving success rates on AI2Thor simulated household tasks with a single-arm, mobile manipulator. Additionally, we implement EmbodiedRAG on a quadruped with a manipulator to highlight the performance benefits for robot deployment at the edge in real environments.

1 Introduction

Exciting recent advances in 3D scene graphs (3DSGs) (e.g., [1, 2, 3, 4]) and large language model (LLM)-based planning (e.g., [5, 6, 7, 8, 9, 10]) are facilitating the expansion of robot task planning and execution to rich, large-scale environments and to complex, long-time horizon tasks. In particular, 3D scene graphs allow for a robot to maintain a rich, semantic, yet compact representation of its operating environment which can then be serialized for use with an LLM-based planner. Thus, with language serving as a bridge, we can ground the LLM-based planner within the robot's operating environment for effective planning leveraging an LLM's strengths (e.g., common sense reasoning).

However, as the robot's environment scales (e.g., from rooms to houses to office buildings) or the complexity of entity information increases (e.g., maintaining more attributes or detailed descriptions per entity), providing the 3DSG as-is to the LLM-based planner presents a couple of challenges. Almost immediately, the 3DSG will take up the token input limit for an LLM (e.g., GPT-4o has a 128k token limit). Even if input token count is not a limitation, reducing the input token count has the important potential to speed up planning time for the robot. Secondly, an LLM's attention capabilities are lacking; LLMs have been shown to be distracted by task-irrelevant information present in input prompts [11, 12] as well as have positional bias to information provided at the beginning or ending of

input prompts [13, 14]. Such limitations make it challenging for the LLM to identify important entities and environmental changes necessary for completing its task. Removing task-irrelevant information also removes associated uncertainty present in the 3DSG from the LLM potentially leading to more robust plans.

To mitigate these challenges, we take inspiration from the successes of Retrieval-Augmented Generation (RAG) methods (see [15] for an overview) that retrieve task-relevant document chunks for LLM question and answering and adapt the paradigm to our embodied domain. Specifically, we seek to retrieve subgraphs from the robot's 3DSG that are task-relevant, i.e., only contain entities, attributes, and relationships sufficient for completing a task. Importantly, the retrieved subgraph should adapt to changes in the environment and changes in task-relevancy. For example, if new, task-relevant information is added to the 3DSG, the subgraph provided to the LLM should incorporate this new information. Additionally, during replanning, the LLM-based planner may realize different entities could help complete the task due to what is present (or missing) in the robot's operating environment (i.e., the retrieved subgraph should incorporate feedback from planner).

Statement of Contributions. Our primary contribution is a 3D scene subgraph retrieval framework, called EmbodiedRAG, that we augment an LLM-based planner with for robotic tasks. Our key innovation is adapting the RAG paradigm for the embodied domains with proposed methods for pre-retrieving task-relevant entities and attributes, indexing a 3DSG, retrieving and grounding to a 3DSG, generating plans, and incorporating planner feedback for retrieval. To our knowledge, this is the first work to extract relevant subgraphs for LLM-based planning that accommodates time-varying scene graphs (e.g., the 3DSG can be constructed on-the-fly), does not require an LLM to parse or manipulate the entire 3DSG for planning, and is agnostic to explicit 3DSG formatting (e.g., hierarchical, open-vocabulary, attribute types). We demonstrate EmbodiedRAG's ability to significantly reduce input token counts and planning time (up to 90% cumulative token use and 70% in average planning step time) while improving planning quality and success rates on AI2Thor simulated household tasks. We additionally implement EmbodiedRAG onboard a quadrupedal platform with a manipulator and compare against an agent using the full 3DSG (which often gets overwhelmed immediately by the token volume).

4 EmbodiedRAG

To extract task-relevant subgraphs, our key insight is to connect our problem to the RAG paradigm. RAG frameworks typically have a document indexing phase for chunking information to be embedded into vector representations, a retrieval phase that brings up top relevant documents related to the user query, and then a generation phase where an LLM answers the query using the retrieved documents [27, 15]. More advanced RAG frameworks additionally use pre-retrieval phases to enhance the quality of the documents retrieved by using strategies such as query transformation

or rewriting [28]. In the following subsections, we discuss how these RAG phases inspire our subgraph extraction strategy $f(\mathcal{G}_t, \tilde{x}_t)$ and how we incorporate feedback mechanisms for operating in real environments (see Figure 1).

4.1 Document Indexing a 3DSG. In order to work in the RAG paradigm, we first need to connect the notion of documents to a 3DSG. Typically, an important consideration in RAG frameworks is deciding how best to chunk document content (e.g., sentences, paragraphs, pages, images). 3DSGs are already chunked into entities by construction, so a natural formulation is to treat each entity o_j as a document to be retrieved. Specifically, we assign the object class or open-vocabulary label as a document's page content. Attributes are assigned as the document metadata. Alternatively, attributes can be treated as sub-documents such that if they are retrieved, the parent document o_j is retrieved. Each document is then embedded in a vector store for efficient retrieval in downstream tasks.

4.2 LLM-Guided Abstractions for Pre-Retrieval. The next RAG component we take inspiration from is the pre-retrieval phase. For our setting, we want to avoid having an LLM parse or traverse an entire 3DSG, and at the start of robot deployment, the 3DSG might not be available yet. Thus, we want a strategy that enhances the quality of retrieved entities as knowledge about them becomes available. We propose generating an entity-oriented abstraction from the natural language task l and the attributes that will be maintained in the 3DSG. Concretely, we ask an LLM prior to planning and any knowledge about the environment to suggest a set of relevant entities that would be useful for the natural language task l. If attributes are expected to be tracked in the 3DSG, then for each entity, we ask an LLM to suggest which attributes are important.

4.3 Subgraph Grounding for Retrieval. As the robot begins deployment in its operating environment $\mathcal{E}$, 3DSG construction begins and each entity is embedded as a document. To construct subgraph $\mathcal{G}'_t$, we first retrieve a thresholded, top-k similarity documents to the pre-retrieved entities. The retrieved entities serve as entry points into our 3DSG, similar to how the local search component in graphRAG gains access to a knowledge graph. Next, we take the induced subgraph from these nodes to capture the relationships between the retrieved entities. We introduce a feedback mechanism, described in Section 4.5, to handle the scenarios when O'_t does not contain all the entities needed to complete the task in environment $\mathcal{E}$.

4.4 Plan Generation. The last main component of a RAG framework is the generation phase where the original query and retrieved documents are provided to an LLM in order to answer the query. Here, we want an LLM to generate actions a_t instead of answers to a query, i.e., serve as our policy $\pi(a_t \mid \mathcal{G}'_t, \tilde{x}_t)$. To do so, we leverage a ReAct-style LLM agent that is designed to generate actions given the history of past actions and observations.

4.5 Incorporating Feedback. Up until now, the retrieval process has been open-loop and following a canonical RAG framework. However, the augmented actions (a_t, τ_t) of the LLM-based planner have

the ability to close feedback loops and affect components in the EmbodiedRAG framework in two ways. Intuitively, action a_t directly affects the environment, which in turn, updates the documents or adds new documents. The second feedback component is via the thoughts τ_t. Although thoughts do not affect the environment, our key insight is to use them to affect our retrieval mechanism as they provide insight into the type of information the LLM-based planner is currently making decisions about. To process the thoughts, we use the self-query method for RAG that uses an LLM to generate a structured query from the original prompt in order to identify important semantics and perform metadata filtering.

7 Conclusion

We developed and evaluated EmbodiedRAG to address the challenges of efficiently integrating 3D scene graphs with LLMs for robot task planning in dynamic, open-world environments. Notably, our approach does not require a 3DSG a priori and it avoids providing an LLM-based planner the entire 3DSG, leading to faster planning generation and improved success rates as compared to a ReAct-only agent and one that receives the full 3DSG. Pivotal to our approach are the feedback components to aid in planning recovery during deployment. Actions suggested by the LLM-based planner affect the environment while the planner's natural language thoughts affect additional, task-relevant retrievals.

Limitations and Future Work. The retrieved entities by EmbodiedRAG are dependent on those that are added to the 3DSG, and thus the performance can be affected by uncertainty involving perception (e.g., multiple entities representing the same object will result in multiple entities retrieved) and localization (e.g., the robot could move to the wrong location relative to a retrieved entity). Future directions can include expanding to multimodal retrieval techniques — an avenue our framework is equipped to extend to. Our framework could also benefit from faster, structured queries for feedback; the self-query mechanism used in this work almost doubled the time per planning step despite improved success rates. In particular, we hope our framework promotes further research in pre-retrieval, retrieval, and self-query components tailored for the embodied domain.

参考文献

[1] Armeni I, et al. 3D scene graph: A structure for unified semantics, 3D space, and camera. ICCV, 2019.
[2] Maggio D, et al. CLIO: Real-time task-driven open-set 3D scene graphs. IEEE RA-L, 2024.
[3] Gu Q, et al. ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning. ICRA, 2024.

[4] Rosinol A, et al. 3D dynamic scene graphs: Actionable spatial perception with places, objects, and humans. arXiv:2002.06289, 2020.
[5] Brohan A, et al. Do as I can, not as I say: Grounding language in robotic affordances. CoRL, 2023.
[6] Rana K, et al. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. CoRL, 2023.
[7] Silver T, et al. Generalized planning in PDDL domains with pretrained large language models. AAAI, 2024.
[8] Liu B, et al. LLM+P: Empowering large language models with optimal planning proficiency. arXiv:2304.11477, 2023.
[9] Yao S, et al. ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2022.
[10] Huang W, et al. Inner monologue: Embodied reasoning through planning with language models. arXiv:2207.05608, 2022.
[11] Shi F, et al. Large language models can be easily distracted by irrelevant context. ICML, 2023.
[12] Oh P, Thorne J. Detrimental contexts in open-domain question answering. arXiv:2310.18077, 2023.
[13] Xiao G, et al. Efficient streaming language models with attention sinks. arXiv:2309.17453, 2023.
[14] Liu N F, et al. Lost in the middle: How language models use long contexts. TACL, 2024.
[15] Gao Y, et al. Retrieval-augmented generation for large language models: A survey. 2024.
[27] Lewis P, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS, 2020.
[28] Zheng H S, et al. Take a step back: Evoking reasoning via abstraction in large language models. arXiv:2310.06117, 2023.
[29] Shao Z, et al. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. arXiv:2305.15294, 2023.
[30] Edge D, et al. From local to global: A graph RAG approach to query-focused summarization. arXiv:2404.16130, 2024.
[36] Chroma. https://github.com/chroma-core/chroma.
[37] Douze M, et al. The FAISS library. arXiv:2401.08281, 2024.
[38] LangChain. https://github.com/langchain-ai/langchain.
[39] LangChain. Self-querying. https://python.langchain.com/docs/how_to/self_query/.
[40] Kolve E, et al. AI2-Thor: An interactive 3D environment for visual AI. arXiv:1712.05474, 2017.